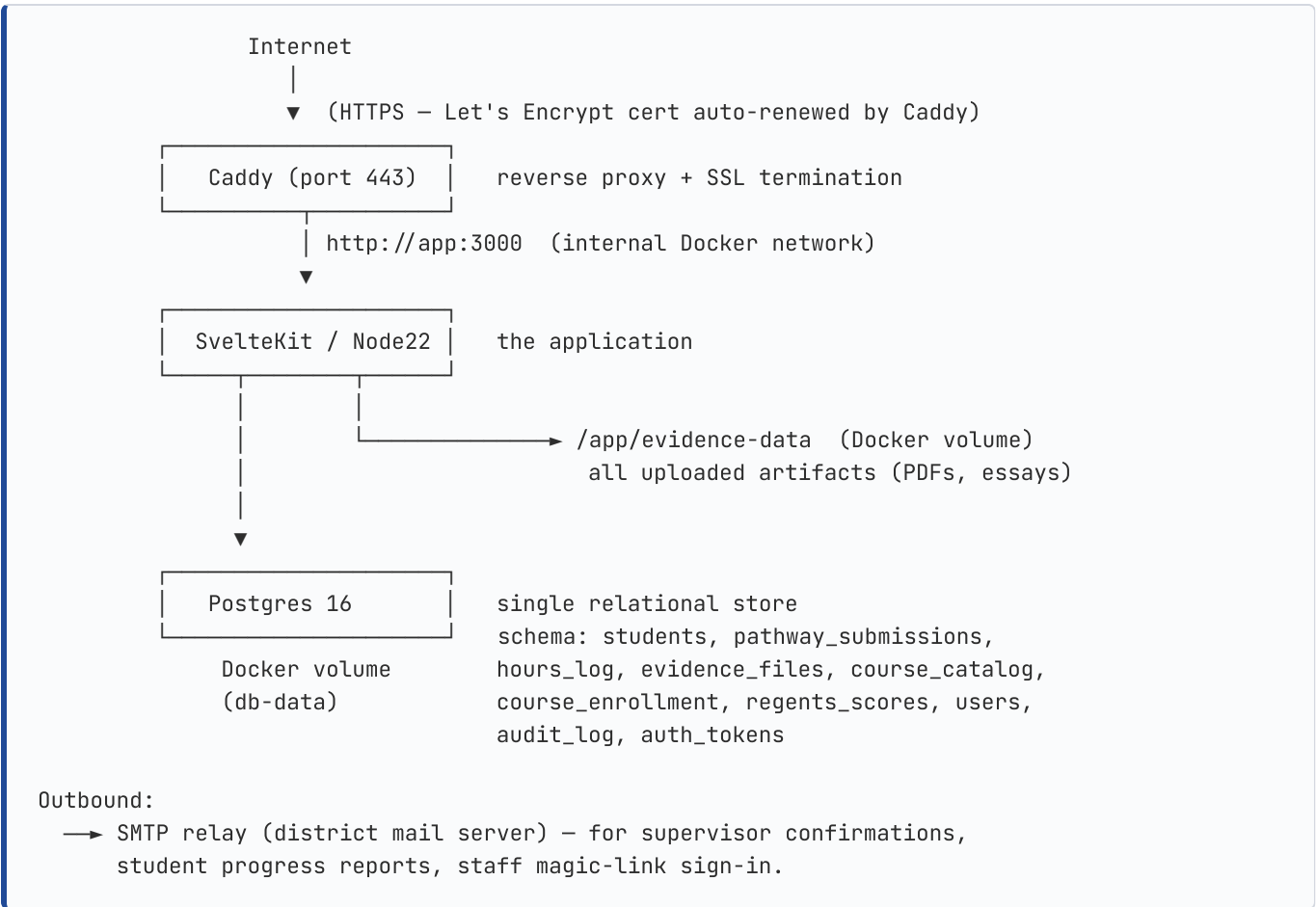


Audience: Great Neck Public Schools Technology Department. **Purpose:** Everything the IT team needs to take this system into production and operate it long-term, on GNPS-owned infrastructure, with no third-party SaaS.

Live demo (current): <https://gnps-civic-readiness.vercel.app> (free Vercel + Supabase tier — superseded once self-host is up) **Source code (MIT):** <https://github.com/SirhanMacx/gnps-civic-readiness>

1. Architecture (self-hosted)



Trust boundary: everything inside the Docker network is private. Only Caddy listens on the public interface. Postgres and the app are not reachable from the internet directly.

Resource footprint at GNPS scale (~6,800 students, ~2,500 submissions/yr):

Component	RAM	CPU	Disk
App (Node)	~512 MB	<0.5 vCPU	none
Postgres	~256 MB	<0.5 vCPU	~50 MB
Caddy	~64 MB	negligible	~50 MB (logs)
Evidence files	n/a	n/a	~1 GB (depends on artifact uploads; bounded by NYSED retention rules)
Total	~1 GB	1 vCPU	~1 GB/yr

A 2 vCPU / 4 GB RAM / 80 GB disk Linux VM has 4× headroom for years.

2. Prerequisites

Server

- Linux (Ubuntu 22.04 LTS or Debian 12 recommended; Rocky / Alma also fine)
- 2+ vCPU, 4+ GB RAM, 50+ GB disk
- Public IPv4 (and ideally IPv6) reachable from the internet on ports 80 and 443
- Docker Engine 24+ and Docker Compose v2 installed
- Outbound network access to:
 - GitHub (code pulls during deploy / updates)
 - Let's Encrypt ACME endpoint (port 443 outbound)
 - Docker Hub (image pulls)
 - District SMTP server

DNS

- An A record (and optional AAAA) for `civicseal.greatneck.k12.ny.us` pointing to the server's public IP, **propagated before first deploy** (Caddy uses HTTP-01 ACME challenge; if DNS isn't resolving, Let's Encrypt will refuse to issue a cert).

Firewall (district-side)

- **Inbound to server:** TCP 80 (HTTP, used only for ACME + redirect to HTTPS), TCP 443 (HTTPS app traffic), TCP 22 (SSH for ops; restrict source IPs to district network).
- **Outbound from server:** TCP 443 (Docker Hub, GitHub, Let's Encrypt, Supabase if migration import is used), TCP 587 or 25 (SMTP to district mail server).

SMTP credentials

From the district mail administrator, you'll need: - SMTP host (e.g. `smtp.greatneck.k12.ny.us`) - Port (usually 587 with STARTTLS; sometimes 465 with implicit TLS) - Authentication credentials (a service account email + password — recommend creating `civicseal-portal` as a dedicated mailbox) - Confirmation that the SMTP server permits sending from `civicseal@greatneck.k12.ny.us` using the provided credentials, OR an alternative `EMAIL_FROM` address that's authorized

Privacy/compliance

- **FERPA review.** Student data lives on GNPS infrastructure (your server). All student record reads/writes are gated by district-issued accounts (counselor / SCRC / admin). The audit_log table records every state transition. Encryption at rest is provided by the underlying disk encryption of the host VM (recommend LUKS or cloud-provider-managed encryption).
- **Backup policy.** See §6.

3. First deployment

Step 1 — provision the server

Whatever your standard procedure is for spinning up a Linux VM. After you SSH in:

```
# As root or a sudo user:
apt update && apt upgrade -y
apt install -y docker.io docker-compose-plugin git make curl
systemctl enable --now docker

# Create a dedicated user for the deploy (optional but recommended)
useradd -m -s /bin/bash civicseal
usermod -aG docker civicseal
su - civicseal
```

Step 2 — clone the repo

```
cd ~
git clone https://github.com/SirhanMacx/gnps-civic-readiness.git
cd gnps-civic-readiness
```

(Once the repo is transferred to a GNPS-owned org, update the URL.)

Step 3 — configure environment

```
cp .env.example .env
nano .env          # or your editor of choice
```

Fill in **all** values. The required ones are:

VARIABLE	EXAMPLE	NOTES
<code>CIVICSEAL_DOMAIN</code>	<code>civicseal.greatneck.k12.ny.us</code>	Must match the DNS A record
<code>POSTGRES_PASSWORD</code>	(32+ char random)	Generate: <code>openssl rand -base64 32</code>
<code>SESSION_SECRET</code>	(32+ char random)	Generate: <code>openssl rand -hex 32</code>
<code>SIGNED_LINK_SECRET</code>	(32+ char random)	Generate: <code>openssl rand -hex 32</code> (different from SESSION_SECRET)
<code>SMTP_HOST</code>	<code>smtp.greatneck.k12.ny.us</code>	From mail admin
<code>SMTP_USER</code>	<code>civicseal-portal</code>	From mail admin
<code>SMTP_PASS</code>	(mailbox password)	From mail admin
<code>EMAIL_FROM</code>	<code>"GNPS Civic Readiness <civicseal@greatneck.k12.ny.us>"</code>	Must be SMTP-authorized

Permissions on the .env file:

```
chmod 600 .env
```

Step 4 — first boot

```
make up
```

This: 1. Builds the SvelteKit app's Docker image (~3 min on first run, cached after that) 2. Starts Postgres 3. Runs the one-shot migration container (applies all SQL migrations to a fresh DB) 4. Starts the app 5. Starts Caddy, which obtains a Let's Encrypt cert for `CIVICSEAL_DOMAIN` (~30–60 sec on first try)

Watch the logs:

```
make logs
# Press Ctrl-C to detach (containers keep running)
```

You'll see Caddy log a successful `obtained certificate` line. After that:

```
curl -I https://civicseal.greatneck.k12.ny.us/health
# expect: HTTP/2 200 ... content-type: application/json
```

Step 5 — first admin user

The portal's auth flow requires a row in `public.users` with `role='admin'` to bootstrap. Create the first admin:

```
make admin EMAIL=jon@greatneck.k12.ny.us
```

Then have Jon visit <https://civicseal.greatneck.k12.ny.us/login>, enter that email, click the magic link from the email he receives. From there he can invite the rest of the staff via </admin/users>.

Step 6 — smoke test

Run through the key user paths: 1. Anonymous visitor hits </> → branded landing page 2. Visitor hits </about> → full pathway breakdown 3. Visitor hits </admin> → redirected to </login?next=/admin> 4. Admin logs in via magic link → lands on </admin>, sees the empty roster (no students yet) 5. Admin visits </admin/import> → uploads <docs/sample-ic-data.csv> → preview shows 5 students + enrollments + Regents → commits → roster shows 4 of 5 students with auto-counted Knowledge points 6. Anonymous visitor hits </submit/service> → fills the form → submits → success banner

If all of those pass, you're live.

4. Operations

Common ops via Make targets

```
make up          # start the stack (idempotent)
make down        # stop the stack (preserves data)
make restart     # restart the stack
make logs        # tail logs from all services
make migrate     # run any pending DB migrations (auto on `up` too)
make shell       # open a shell inside the app container
make db-shell    # open psql against the running DB
make admin EMAIL=alice@greatneck.k12.ny.us
make status      # show service status + healthchecks
make build-image # rebuild the app Docker image without cache (after big changes)
make clean       # DESTRUCTIVE: stop + remove containers AND volumes (data loss)
```

Logs

All services log to stdout, captured by Docker:

```
docker compose logs -f --tail=200 app  # app logs (HTTP requests, app errors, audit events)
docker compose logs -f --tail=200 db   # Postgres logs (connection errors, slow queries)
docker compose logs -f --tail=200 caddy # access log (one JSON line per request) + cert
renewal events
```

For long-term log retention (recommended after pilot phase): - Configure Docker's logging driver to ship to your district SIEM (`/etc/docker/daemon.json` with `log-driver: syslog` or `json-file` with size limits) - Or run `vector` / `fluent-bit` as a sidecar to ship logs elsewhere

Health checks

The app exposes `GET /health` returning JSON `{ status: "ok", service, timestamp }`. Caddy proxies it; you can hit it externally for uptime monitoring:

```
curl https://civicseal.greatneck.k12.ny.us/health
```

For deeper health, the migration container exits non-zero if the DB is unreachable on boot. Set up your monitoring to alert on: - `app` container not running for >5 min - `db` container not running for >2 min - HTTP `/health` returning non-200 for >3 consecutive checks - Disk usage on the host >85%

5. Updates

The application updates by pulling the latest tagged release and rebuilding the image:

```
cd ~/gnps-civic-readiness
git fetch --tags
git checkout v0.2.0    # or whatever tag you're moving to
make build-image      # rebuilds with --no-cache
make up               # restarts containers; migration runner auto-applies any new SQL
```

Schema migrations are idempotent. The migration runner refuses to re-apply an already-applied file with a different content hash, so accidental edits are caught early.

Zero-downtime updates are not yet wired (Phase 1.5 of self-host). For now, plan for a 30–60 second outage during `make up`. Schedule updates outside school hours.

Rolling back to a previous version:

```
git checkout v0.1.0
make build-image && make up
```

Note: this works as long as no schema migrations were applied that the older code can't handle. If you rolled forward through a destructive schema change, restoring from a DB backup is the rollback path (see §6).

6. Backups + Disaster Recovery

What to back up

Two persistent volumes: - `db-data` — the Postgres data directory - `evidence-data` — the uploaded student artifacts (PDFs, essays)

Backup procedure

Daily — automated via cron on the host:

```
# /etc/cron.d/civicseal-backup — runs every night at 2am
0 2 * * * civicseal /home/civicseal/gnps-civic-readiness/scripts/backup.sh
```

Create `scripts/backup.sh`:

```
#!/usr/bin/env bash
set -euo pipefail

BACKUP_DIR=/var/backups/civicseal
DATE=$(date +%Y-%m-%d-%H%M)
mkdir -p "$BACKUP_DIR"

# 1. Postgres logical dump (pg_dump runs inside the db container)
docker compose exec -T db pg_dump -U civicseal civicseal | gzip > "$BACKUP_DIR/db-$DATE.sql.gz"

# 2. Evidence files (incremental rsync to backup volume; or tar for full dump)
docker run --rm \
  -v civicseal_evidence-data:/source:ro \
  -v "$BACKUP_DIR":/backup \
  alpine tar -czf "/backup/evidence-$DATE.tar.gz" -C /source .

# 3. Retention: keep daily for 30 days, weekly for 12 weeks, monthly for 12 months
find "$BACKUP_DIR" -name "db-*.sql.gz" -mtime +30 ! -name "db-*-Sun-*" -delete
find "$BACKUP_DIR" -name "evidence-*.tar.gz" -mtime +30 ! -name "evidence-*-Sun-*" -delete

# 4. Off-host copy (highly recommended): rsync to a secondary machine
# rsync -az --delete "$BACKUP_DIR/" backup-host:/backups/civicseal/
```

Make it executable: `chmod +x scripts/backup.sh`.

Weekly — verify restore works: 1. Spin up an empty Postgres container on a test host 2. Pipe the latest dump in: `gunzip -c db-LATEST.sql.gz | psql -h localhost -U postgres testdb` 3. Verify table row counts roughly match prod 4. Stop the test container

If you can't restore the backup, the backup isn't real.

Off-site copy: the `db-data` and `evidence-data` directories should be replicated to a second physical location (district secondary site, cloud bucket, etc.). Talk to whoever owns district backup policy.

Restore procedure

If the worst happens and you need to restore from scratch:

```
# 1. Provision a fresh server, install Docker + Compose, clone the repo, copy .env
# 2. Bring the stack up with --no-deps so we can restore manually:
docker compose up -d db
# 3. Restore the DB
gunzip -c /path/to/db-BACKUP.sql.gz | docker compose exec -T db psql -U civicseal civicseal
# 4. Restore evidence files
docker run --rm \
  -v civicseal_evidence-data:/dest \
  -v /path/to/backups:/backup \
  alpine tar -xzf /backup/evidence-BACKUP.tar.gz -C /dest
# 5. Bring up the rest
make up
```

7. Security checklist

ITEM	FREQUENCY	OWNER
Rotate <code>SESSION_SECRET</code> and <code>SIGNED_LINK_SECRET</code>	Annually, or after suspected leak (signs everyone out)	IT
Rotate Postgres password	Annually (requires app restart)	IT
Rotate SMTP service account password	When district policy requires	IT
Review <code>users</code> table — remove staff who no longer need access	Quarterly	C&I + IT
Review <code>audit_log</code> for unusual activity	Monthly	C&I
Apply Linux security updates on the host	Weekly (<code>apt upgrade -y</code>)	IT
Apply Docker engine + Compose updates	Quarterly	IT
Apply Postgres image updates (<code>postgres:16-alpine</code> is the floating tag — pin to a specific digest in <code>docker-compose.yml</code> if your policy requires)	When upstream releases a security patch	IT
Re-pull app image after a tagged release	When a new GNPS Civic Readiness Portal version ships	IT
Verify backup restore works	Weekly	IT
Re-run the smoke-test checklist	After every deploy	IT + C&I

Secret hygiene: - The `.env` file at the repo root contains all secrets. `chmod 600 .env`. Don't commit it. Don't paste it in chat or email. - The `_app/immutable/...` URLs the browser sees are fingerprinted; changing `SESSION_SECRET` invalidates all sessions but doesn't expose anything to clients.

8. Troubleshooting

Caddy won't issue an SSL certificate

Symptom: `make logs` shows Caddy looping with `obtaining certificate failed` **Fix:** 1. Confirm DNS: `dig civicseal.greatneck.k12.ny.us` → must return your server's IP 2. Confirm port 80 is reachable: from outside the network, `curl -I http://civicseal.greatneck.k12.ny.us` should hit Caddy and redirect (not be blocked by firewall) 3. Wait. Let's Encrypt rate-limits per domain — if you've spammed the cert request, wait an hour.

App container restart loops

Symptom: `make status` shows `app` repeatedly going down **Fix:** `make logs app | tail -100` — usually one of: - `DATABASE_URL` is wrong → fix in `.env`, `make restart` - `SESSION_SECRET` is shorter than 32 chars → fix in `.env`, `make restart` - DB hasn't finished migrating yet → `make migrate` then `make restart`

Emails aren't being sent

Symptom: Students submit but the supervisor never gets a confirmation email **Fix:** 1. `make logs app | grep -i smtp` — if you see "SMTP not configured", `.env` is missing SMTP_HOST/USER/PASS 2. If SMTP is configured but emails aren't arriving: - Test SMTP creds from inside the container: `make shell` → `nc smtp.greatneck.k12.ny.us 587` should connect - Check the district mail admin's logs for blocked/quarantined messages - Check the spam folder

"Email not found" on /login

Symptom: Staff member tries to log in but gets "Email not found — ask an admin to invite you" **Fix:** Their email isn't in `public.users`. Either: - Use `make admin EMAIL=...` to add them as admin - Have an existing admin invite them via `/admin/users` with the appropriate role

Disk fills up

Symptom: Container errors include "no space left on device" **Fix:** 1. Check evidence-data size: `du -sh /var/lib/docker/volumes/civicseal_evidence-data/_data` 2. Old NYSED audit packs and evidence files can be archived off the live server after the relevant graduation cohort ages out (typically 7 years per district records-retention policy — confirm with C&I) 3. Postgres also grows; `make db-shell` then `\dt+` shows table sizes. The `audit_log` table is the largest by row count; you can archive rows older than 7 years if district policy permits

“Can’t connect to database” on app boot

Symptom: App container exits with `FATAL: connection refused` **Fix:** `make status` – if `db` is unhealthy, check `docker compose logs db` for the underlying issue (corrupted data, version mismatch, etc.). For a clean DB restart: `docker compose restart db`. If that fails, restoring from backup is the path.

9. Operational schedule

FREQUENCY	TASK
Daily	Automated DB + evidence backup (cron); Caddy auto-renews certs (90-day Let’s Encrypt cycle)
Weekly	Verify the latest backup restores into a test DB; review <code>audit_log</code> for unexpected actions
Monthly	Apply OS security patches; review staff <code>users</code> table; confirm SMTP is delivering
Quarterly	Reach out to C&I to confirm the SCRC committee membership matches reality; offboard any staff who left the district
Annually	Rotate <code>SESSION_SECRET</code> + <code>SIGNED_LINK_SECRET</code> ; rotate Postgres password; rotate SMTP creds; review NYSED rules for any updates that affect pathway logic
At each NYSED Seal Manual update	Review new criteria; confirm <code>packages/pathway-rules/</code> reflects current rules; bump version and redeploy

10. Capacity & scale

GNPS scale (~6,800 students, ~412 per graduating class, ~2,500 active submissions per year) fits comfortably on the spec’d 2 vCPU / 4 GB / 50 GB box for the lifetime of the program.

When to consider scaling up: - Sustained load >50% on a single vCPU during peak hours (May submission window) → increase to 4 vCPU - DB size approaches 5 GB → review for archival candidates (`audit_log` older than 7 years, awarded students older than 10 years) - Evidence-data approaches 50 GB → archive completed cohorts to cold storage (move tar.gz of one cohort’s PDFs to long-term storage; keep only the audit-pack PDF in live storage)

If you ever need a horizontal scale (multiple app containers behind a load balancer), the architecture supports it cleanly: stateless app containers + shared Postgres + shared evidence volume (or shared S3 bucket via `STORAGE_BACKEND=s3`). Talk to me / the maintainers when that day comes.

11. Decommissioning

If the district ever sunsets the program (or migrates to a vendor), here's how to retire the system cleanly without losing student records (NYSED retention rules apply):

1. **Notify staff and students** ~60 days in advance. Stop accepting new submissions.
2. **Run the year-end NYSED audit-pack export** for every active cohort: `make admin-shell` (or via `/admin/export?cohort=YYYY` for each year)
3. **Archive the full DB dump and the evidence-data tarball** to permanent district storage — these are FERPA records and must be retained for the district records-retention policy duration (typically 7 years)
4. **Document the retirement** in the audit_log via a one-shot script (so future audits show a clean shutdown, not a data leak)
5. **Stop the stack:** `make down`
6. **After retention period expires**, securely wipe the volumes: `make clean` (DESTRUCTIVE)

12. Contacts

ROLE	PERSON	CONTACT
Program lead (Social Studies)	Jon	civicseal@greatneck.k12.ny.us
C&I sponsor	(TBD)	(TBD)
Technical maintainer	(initially Jon, then GNPS IT)	civicseal@greatneck.k12.ny.us
Repository	github.com/SirhanMacx/gnps-civic-readiness	until transferred
NYSED contact	regional Seal of Civic Readiness coordinator	(see NYSED handbook)

13. Appendices

- **A — Architecture diagram (high-res):** `docs/architecture-self-hosted.md`
- **B — Infinite Campus integration:** `docs/infinite-campus-integration.md`
- **C — Customization for non-GNPS districts:** `docs/customization.md`
- **D — Data import format:** `docs/data-import-guide.md`
- **E — Original design document:** `dist/GNPS-Civic-Readiness-Portal-Design.pdf` (27 pages)
- **F — IT-handoff brief (executive summary):** `dist/GNPS-IT-Handoff-Brief.pdf` (6 pages)